

[OTA] Technical specs for Master OTA

[Introduction](#) | [Reaching from initial to ready state \(startup\)](#) | [Resumable Download Handling](#) | [1. Download Initiation](#) | [2. Persistent Download State](#) | [3. Resume on Restart or New Driving Cycle](#) | [Handling New Package Versions](#) | [Post-Download Validation](#) | [1. Package Completeness Verification](#) | [2. Package Integrity and Authenticity Verification](#) | [Package Transfer to Eligible OTA Agents](#) | [Manifest-Based Compatibility Checks](#) | [Agent-Side Receipt and Acknowledgement](#) | [Rollback and Success Handling Strategy](#) | [Status Reporting from OTA Agent](#) | [Update Monitoring, Success, and Rollback Handling](#) | [Update Failure and Agent-Driven Rollback](#) | [Completion Criteria](#)

Introduction:

As described in the scope document, this node is responsible for receiving update packages, validating their compatibility, determining the target CPUs for which an update is applicable, and performing other related coordination tasks. This document further elaborates on the internal design and the functional responsibilities through which these tasks are executed.

In the ideal deployment model, a single instance of this service runs on the primary HPC. This instance acts as the central authority for the OTA process, ensuring that updates are downloaded, installed, and activated correctly. In addition, it continuously monitors system health by performing audits, validating newly updated services in a sandboxed environment, and managing rollback operations when required (for example, reverting to a previously known-good version).

Caveat:

There may be scenarios in which control of the master OTA service must be temporarily transferred to a secondary HPC agent. This situation typically arises when a firmware update is being applied to the primary HPC on which the master service is currently running (if such a failover mechanism is enabled). In this case, prior to activation, the master service will notify the OTA agent on the secondary HPC to assume temporary control of the system. Once the primary HPC successfully reboots and resumes normal operation, the secondary agent will relinquish its elevated privileges and return control to the primary service.

Reaching from initial to ready state (startup)

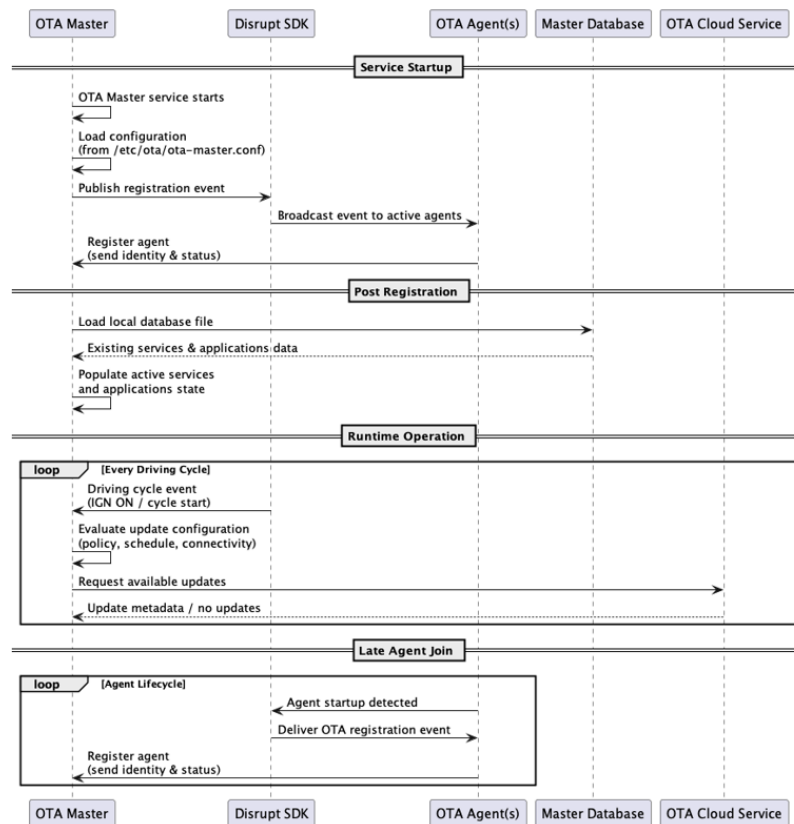
Upon service startup, the **OTA Master** loads its configuration from the local configuration file (for example, `/etc/ota/ota-master.conf`). Once initialized, the OTA Master publishes a **registration event** via **DisruptSDK**, which is broadcast to all currently active OTA agents in the system. Each active agent, upon receiving this event, deterministically registers itself with the OTA Master by providing its identity, status, and capabilities.

The OTA Master then completes its initialization by loading the local database file and populating the list of existing and active services and applications currently running on the system.

The registration mechanism is **event-driven, persistent, and deterministic**. If an OTA agent becomes active *after* the OTA Master has already started, the same registration event mechanism is reused. Newly started agents receive the registration event via DisruptSDK and deterministically register themselves with the OTA Master without requiring a restart or special handling.

During normal runtime, the OTA Master operates in strict alignment with the vehicle lifecycle. On **every driving cycle**, a driving-cycle event (for example, IGN ON) is delivered to the OTA Master via DisruptSDK. Based on this event and the configured policies (such as schedule, connectivity, and update rules), the OTA Master deterministically evaluates whether an update check should be performed. If applicable, it requests available update metadata from the OTA cloud service.

All decision points, including agent registration, update checks, and cloud interactions, follow **deterministic rules derived from configuration and vehicle state**, ensuring predictable behavior across reboots, driving cycles, and system restarts.



Resumable Download Handling

The OTA Master supports deterministic and resumable downloads to ensure robustness against power loss, connectivity interruptions, and driving-cycle boundaries.

1. Download Initiation

When a new package update is identified from the OTA cloud service, the OTA Master evaluates the download eligibility based on configuration and vehicle state.

For the gasoline cars, downloads are only initiated or resumed when the vehicle is in the engine start state (for example, IGN ON). Download operations are not performed outside this state to ensure predictable power and network availability but this behaviour will also be controlled by the configurations by OEM.

Network selection, bandwidth usage, and throttling behavior are governed by OEM-provided configuration policies, which may include (but are not limited to):

1. Allowed network types
2. Bandwidth limits and throttling rules
3. Time- or region-based network policies

If all configured conditions are satisfied, the OTA Master initiates the download process in the background.

Each download is uniquely identified using a **package identifier and version**, derived from the update manifest. This identifier is used consistently across all download attempts and resumes.

2. Persistent Download State

For every active download, the OTA Master maintains a persistent download state, stored locally (for example, under `/var/lib/ota/downloads/`). The persisted state includes, but is not limited to:

- Package identifier and version
- Total package size
- Downloaded byte ranges or completed chunks

- Checksum or hash for completed segments
- Target storage location (inactive slot or staging area)
- Download status (in-progress, paused, completed, failed)

This state is written atomically to ensure consistency across reboots or unexpected shutdowns.

3. Resume on Restart or New Driving Cycle

On OTA Master startup, or at the beginning of each driving cycle, the OTA Master scans the persisted download state to determine whether there are incomplete or paused downloads.

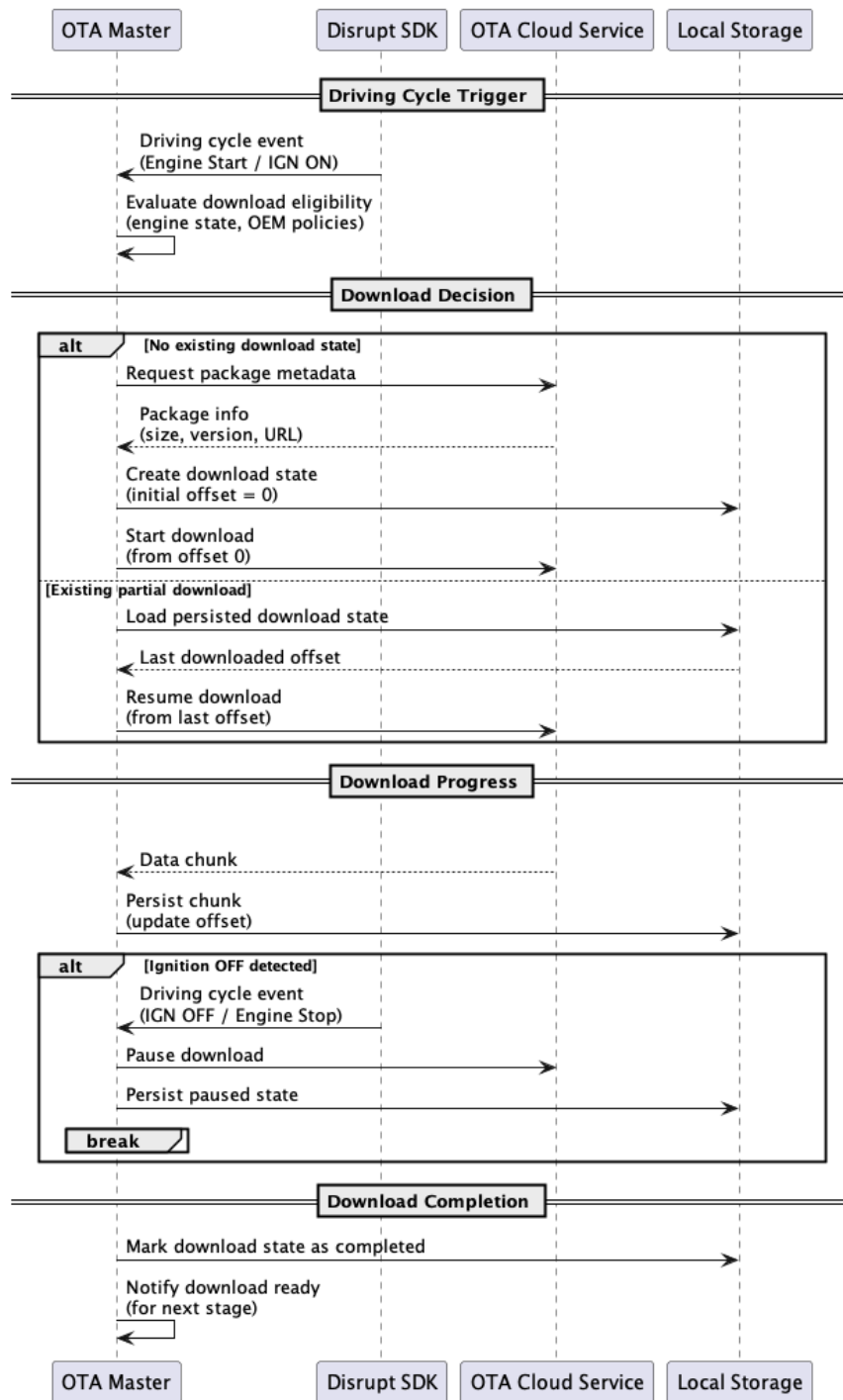
If an incomplete download exists and the vehicle is again in the engine start state, the OTA Master deterministically re-evaluates the OEM-provided network and bandwidth policies. If permitted, the download is resumed from the last successfully completed byte offset or chunk boundary.

The resume operation uses standard **HTTP range requests** or an equivalent chunk-based protocol supported by the OTA cloud service.

Handling New Package Versions

If a newer version of a package is detected while an older version is partially downloaded, the OTA Master deterministically resolves the situation based on configured policy:

- If the newer version supersedes the older one, the in-progress download is cleanly stopped, its state is archived or discarded, and a new resumable download is initiated for the newer package.
- If parallel versions are allowed, each package maintains an independent download state and storage area.



Post-Download Validation

After a package download is completed, the OTA Master performs a post-download verification phase before the package is accepted for installation or activation.

This phase verifies that the downloaded package is **complete, uncorrupted, and originates from the trusted OTA cloud**.

Verification is performed before any transfer, installation, or activation step and does not affect the currently active system.

1. Package Completeness Verification

The first step ensures that the downloaded package is complete and free from transmission or storage errors.

Size Verification:

- The OTA Master compares the size of the downloaded package with the expected size provided by the cloud backend in response.
- Any size mismatch indicates an incomplete or truncated download and results in rejection of the package.

b. CRC32 Verification:

- A CRC32 checksum is computed over the downloaded package.
- The computed CRC32 value is compared with the CRC32 value provided by the cloud backend in response.

2. Package Integrity and Authenticity Verification

After completeness verification succeeds, the OTA Master validates that the package originated from the trusted OTA cloud and has not been modified.

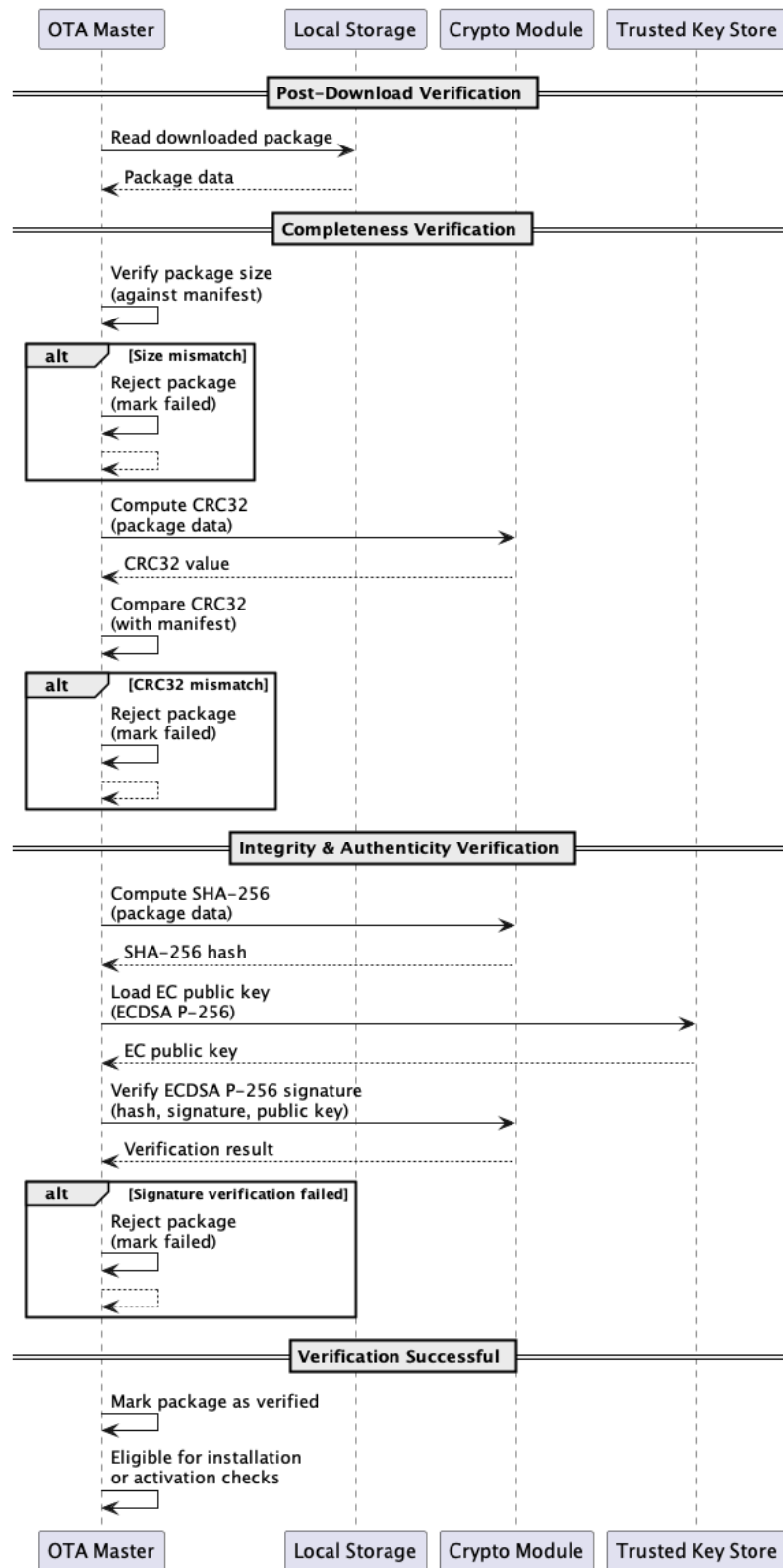
a. Hash Calculation

- A cryptographic hash (SHA-256) of the downloaded package is calculated.
- This hash represents the exact package content to be verified.

b. ECDSA P-256 Signature Verification

- The cloud backend will provide a digital signature generated by the OTA cloud using ECDSA with the P-256 curve.
- The OTA Master verifies this signature using the corresponding trusted EC public key provisioned on the vehicle (the rotation process can be added if we make it part of scope).
- Successful verification confirms:
 - The package was signed by an authorized OTA cloud entity
 - The package content has not been altered since signing

If signature verification fails, the package is rejected.



Package Transfer to Eligible OTA Agents

After a package has successfully passed post-download verification, the OTA Master evaluates whether the package should be transferred to one or more OTA agents. Package transfer is strictly manifest-driven and conditioned on safety, hardware compatibility, and system state.

Before initiating any transfer, the OTA Master evaluates each registered OTA agent against the package manifest and runtime conditions.

Manifest-Based Compatibility Checks

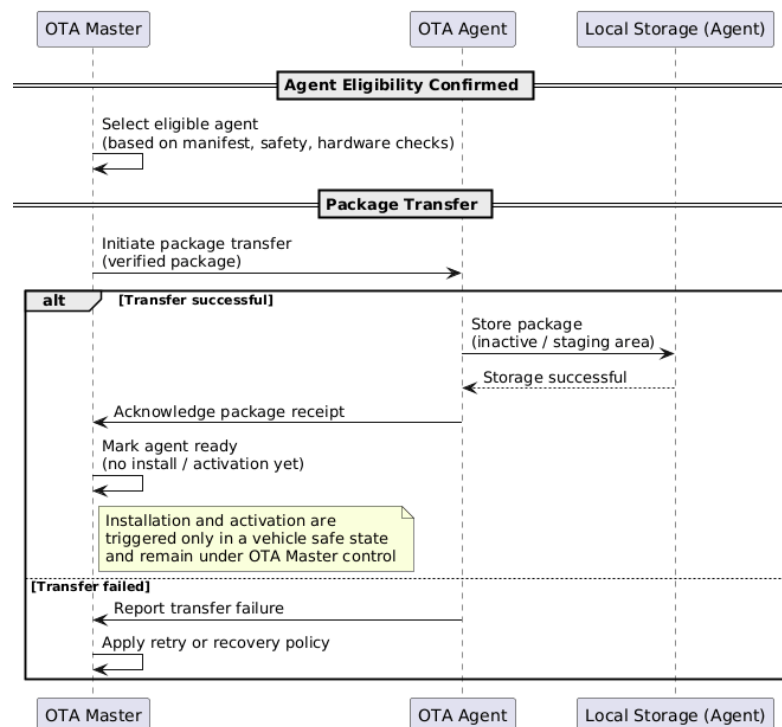
For each agent, the OTA Master validates:

- Hardware compatibility
 - ECU or node identifier
 - Hardware revision or platform variant
 - Architecture or SoC compatibility
- Software compatibility
 - Target component or application identifier
 - Minimum and maximum supported software versions
- Vehicle configuration constraints
 - Region, market, or feature flags
 - OEM-specific enablement rules

Agent-Side Receipt and Acknowledgement

Upon receiving the update package, the OTA agent stores it in its designated inactive or staging area without affecting the currently running software. After the package has been successfully stored, the agent sends an acknowledgment to the OTA Master confirming successful receipt. At this stage, no installation or activation is performed; all such actions remain explicitly controlled and authorized by the OTA Master (activation for safety or non-safety applications will be triggered in vehicle's safe state only).

If the agent reports a failure during the transfer process, the OTA Master handles the situation by applying the configured retry or recovery behavior, ensuring controlled and policy-driven handling of transfer failures.



Rollback and Success Handling Strategy

After the OTA Master initiates installation or activation on an OTA agent, the agent reports its execution status back to the OTA Master. Based on this status, the OTA Master determines whether the update is considered successful or whether rollback actions are required.

While the decision and coordination are handled by the OTA Master, the actual rollback of software or firmware is performed locally by the OTA agent.

Status Reporting from OTA Agent

Each OTA agent reports status to the OTA Master at defined milestones, including:

- Installation started
- Installation completed
- Activation started
- Activation completed
- Failure (with reason code)

Status messages include sufficient context to identify the target package, agent, and execution phase.

Update Monitoring, Success, and Rollback Handling

After an update is installed and activated on an OTA agent, the OTA Master does not immediately mark the update as completed. Instead, it monitors the health of the updated software or firmware across multiple driving cycles.

During each driving cycle, the OTA Master evaluates health and status signals reported by the agent, such as error states, crashes, or degraded behavior. These signals are aggregated over time to assess update stability. If the error rate or fault indicators increase beyond a configured threshold, the OTA Master determines the next action based on the severity and criticality of the updated application or firmware.

For non-critical applications, the OTA Master may allow continued monitoring or defer corrective action. For safety-critical or high-severity components, the OTA Master may instruct the agent to initiate a rollback.

Update Failure and Agent-Driven Rollback

If an update fails during installation or activation, the OTA agent performs a local rollback immediately, without waiting for further instructions from the OTA Master. The rollback is executed using the agent's supported mechanism (for example, reverting to the previous software version or switching back to the active slot).

After completing the rollback, the OTA agent notifies the OTA Master of:

- The failure condition
- The rollback action taken
- The final recovery status

The OTA Master then updates the global update state, records the failure, and applies any retry, blocking, or escalation policy as configured.

Completion Criteria

An update is marked as successfully completed by the OTA Master only when:

- Installation and activation have completed successfully, and
- The update remains healthy across the configured number of driving cycles, and
- No error thresholds are exceeded during the monitoring period

This approach ensures that updates are validated not only at activation time, but also during real vehicle operation, providing controlled recovery and long-term stability.

